Since you're already comfortable with **JavaScript Promises** and **async/await**, Python's asynchronous programming will feel very familiar. The biggest difference is **what is awaitable** and **which event loop library you're using** (usually `asyncio`).

I'll explain everything by mapping Python concepts to JavaScript.

---

# JavaScript vs Python

| JavaScript | Python |
|---|---|
| Promise | Coroutine / Future / Task |
| async function | async def |
| await | await |
| Promise.all() | asyncio.gather() |
| setTimeout() | asyncio.sleep() |
| fetch() | aiohttp |
| Event Loop | asyncio Event Loop |
| then() | await |

---

# 1. Normal Function

JavaScript

```
function hello() {
    return "Hello";
}

console.log(hello());
```

Python

```
def hello():
    return "Hello"

print(hello())
```

Nothing new yet.

---

# 2. Async Function

JavaScript

```
async function hello() {
    return "Hello";
}
```

Notice:

This actually returns a **Promise**.

```
const p = hello();

console.log(p);
// Promise { "Hello" }
```

Python

```
async def hello():
    return "Hello"
```

Calling it

```
result = hello()

print(result)
```

Output

```
<coroutine object hello at ...>
```

Notice something important.

Unlike JavaScript,

Python **does not automatically execute** the async function.

It only creates a **coroutine object**.

Think of it like:

> JavaScript async function → Promise already scheduled

vs

> Python async function → Recipe waiting to be executed.

---

# 3. Await

JavaScript

```
async function hello() {
    return "Hello";
}

async function main() {
    const value = await hello();
    console.log(value);
}

main();
```

Python

```
import asyncio

async def hello():
    return "Hello"

async def main():
    value = await hello()
    print(value)
```

```
asyncio.run(main())
```

Output

```
Hello
```

---

# Why asyncio.run()?

---

This starts the **event loop**.

Equivalent mental model:

```
Node.js starts event loop automatically.
```

```
Python requires you to start it.
```

Think:

```
Node
```

```
Program starts
↓
```

```
Event Loop already running
↓
```

```
Promises execute
```

Python

```
Program starts
↓
```

```
No Event Loop
```

```
↓
```

```
asyncio.run()
```

```
↓
```

```
Event Loop starts
```

↓

```
Coroutines execute
```

---

# 4. Sleeping

JavaScript

```
await new Promise(resolve =>
    setTimeout(resolve, 1000)
);
```

Python

```
await asyncio.sleep(1)
```

Notice

```
sleep(1)
```

means

```
1 second
```

not milliseconds.

---

# Example

```
import asyncio

async def hello():
    print("Start")

    await asyncio.sleep(2)
```

```
    print("End")

asyncio.run(hello())
```

Output

```
Start

(wait 2 sec)

End
```

---

# 5. Multiple Async Operations

JavaScript

```
async function task(id) {
    console.log(id);
    await sleep(1000);
    console.log(id + " done");
}
```

Python

```
async def task(id):
    print(id)
    await asyncio.sleep(1)
    print(id, "done")
```

---

# Sequential Execution

JavaScript

```
await task(1);
await task(2);
await task(3);
```

Python

```
await task(1)
await task(2)
await task(3)
```

Output

```
1
(wait)

1 done

2
(wait)

2 done

3
(wait)

3 done
```

Total time

```
3 seconds
```

---

# Concurrent Execution

JavaScript

```
await Promise.all([
    task(1),
    task(2),
```

```
    task(3)
]);
```

Python

```
await asyncio.gather(
    task(1),
    task(2),
    task(3)
)
```

Output

```
1
2
3
```

```
(wait)
```

```
1 done
2 done
3 done
```

Total

```
1 second
```

Exactly like Promise.all().

---

# 6. Return Values

JavaScript

```
const values = await Promise.all([
    getUser(),
    getPosts()
]);
```

Python

```
users, posts = await asyncio.gather(
    get_user(),
    get_posts()
)
```

Or

```
results = await asyncio.gather(
    get_user(),
    get_posts()
)

print(results)
```

Output

```
[
    user,
    posts
]
```

---

# 7. Error Handling

JavaScript

```
try {
    await fetch(...)
}
catch(err){
}
```

Python

```
try:
    await some_function()
```

```
except Exception as e:
    print(e)
```

Nothing surprising.

---

# 8. Promise → Task

JavaScript

```
const promise = fetch(url);
```

The request immediately starts.

Python

Coroutines do **not** start automatically.

```
coro = fetch()
```

Nothing happens.

To actually schedule it:

```
task = asyncio.create_task(fetch())
```

Now it starts running.

This is extremely important.

---

# Example

```
import asyncio

async def worker():
```

```
    print("Started")

    await asyncio.sleep(2)

    print("Finished")
```

This does nothing:

```
worker()
```

This executes:

```
asyncio.create_task(worker())
```

---

# 9. create_task()

JavaScript

```
const promise = fetch(url);

// fetch already running
```

Python

```
task = asyncio.create_task(fetch())
```

Now

```
Task
↓
Event Loop
↓
Coroutine executes
```

---

# 10. Awaiting Later

JavaScript

```
const p1 = fetch(url1);
const p2 = fetch(url2);

const r1 = await p1;
const r2 = await p2;
```

Python

```
t1 = asyncio.create_task(fetch1())
t2 = asyncio.create_task(fetch2())

r1 = await t1
r2 = await t2
```

Both started immediately.

---

# 11. Producer Consumer Example

```
import asyncio

async def producer():
    for i in range(5):
        print("Produced", i)
        await asyncio.sleep(1)

async def consumer():
    while True:
        print("Consuming...")
```

```
        await asyncio.sleep(0.5)
```

Run together

```
await asyncio.gather(
    producer(),
    consumer()
)
```

Both run cooperatively.

---

# 12. Async HTTP Request

JavaScript

```
const response = await fetch(url);

const json = await response.json();
```

Python

```
import aiohttp

async with aiohttp.ClientSession() as session:

    async with session.get(url) as response:

        data = await response.json()
```

The `async with` statement ensures resources (like network connections) are cleaned up correctly after use.

---

# 13. Async File I/O

Standard Python file I/O is synchronous:

```
with open("file.txt") as f:
    text = f.read()
```

For asynchronous file operations, use a library such as `aiofiles`:

```
import aiofiles

async with aiofiles.open("file.txt") as f:
    text = await f.read()
```

---

# 14. Async Iteration

If an object produces values asynchronously, use `async for`:

```
async for item in stream:
    print(item)
```

This is commonly used with streaming APIs, asynchronous database cursors, and WebSocket messages.

---

# JavaScript → Python Cheat Sheet

| JavaScript | Python |
| --- | --- |
| `async function f() {}` | `async def f():` |
| `await` | `await` |
| `Promise` | Coroutine / Task |
| `Promise.all()` | `asyncio.gather()` |
| `Promise.race()` | `asyncio.wait(..., return_when=asyncio.FIRST_COMPLETED)` |

| JavaScript | Python |
| --- | --- |
| `setTimeout()` | `asyncio.sleep()` |
| `fetch()` | `aiohttp.ClientSession().get()` |
| `new Promise()` | Usually unnecessary; define an `async def` or create a `Future` for advanced cases |
| `then()` | `await` (preferred) |
| `setInterval()` | A loop with `await asyncio.sleep(...)` |
| `AbortController` | `task.cancel()` and `asyncio.CancelledError` |

---

# Mental Model

Think of the Python async ecosystem like this:

```
async def
    |
    ▼
Creates a Coroutine
    |
    ▼
asyncio.create_task()
(or await directly)
    |
    ▼
Event Loop (asyncio.run)
    |
    ▼
Coroutine executes
    |
    ▼
await
    |
    ▼
Coroutine pauses
    |
```

```
      ▼
Event Loop runs other tasks
      |
      ▼
Paused coroutine resumes
```

The key conceptual difference from JavaScript is that **calling an `async def` function does not start it**. It merely creates a coroutine object. The coroutine begins executing only when you `await` it or schedule it with `asyncio.create_task()`. Once you internalize that distinction, the rest of Python's `asyncio` model maps very naturally to the `async` / `await` and Promise patterns you already know from JavaScript.